
COMP4601 Project Documentation by Team HoG Riders

This document covers parts of the project that we could not fit into the presentation and demonstration. Since the presentation already introduced the HoG algorithm and showed the final results, this report focuses on how the implementation was designed, why we made each optimisation, and how we checked that the design stayed correct. It assumes the reader has already seen the presentation and demonstration.

What the 288 Features Represent

(a) A feature is one number that describes part of the image. The image is split into 8x8 pixel cells, while the 0 to 180 degree range of edge directions is split into 8 bins. For each cell, the edge strengths of pixels pointing in the same direction are added into the corresponding bin. The resulting values are then normalised so that the descriptor is less affected by overall image brightness.

(b) Raw pixel values are very sensitive to lighting, contrast and small shifts in the image. A 32x32 image also contains 1,024 pixel values, which is more information than the classifier needs for recognising shape. HoG reduces the image to a 288-value vector that mainly describes edge structure rather than absolute brightness. This is useful for distinguishing shapes such as a handwritten 3 and 8. It also means two images of the same digit can still produce similar features even if one is darker.

(c) The 288 features come from the layout of the descriptor. Each pixel first gets a gradient magnitude and direction. The 32x32 image is then divided into 8x8 pixel cells, giving a 4x4 grid of 16 cells. Within each cell, the 64 pixel gradients are accumulated into 8 orientation bins. The cells are then grouped into overlapping 2x2 blocks, with the block moving by one cell each time. This gives 9 block positions, and each block contains 4 cells with 8 values each. Each block is normalised independently, giving the final $9 \times 4 \times 8 = 288$ features.

Stage	Result	Count
32x32 pixel image	angle and magnitude per pixel	1,024 pixels
Divide into 8x8 pixel cells	4x4 grid of cells	16 cells
8 orientation bins per cell	64 pixels collapse to 8 values	8 per cell
Group into 2x2 cell blocks, sliding by one cell	3x3 grid of block positions	9 blocks
Each block holds 4 cells of 8 bins	normalised independently	32 per block
Concatenate all blocks	final feature vector	$9 \times 32 = 288$

Sizing the Fixed-Point Types

(a) The kernel uses fixed-point types declared in `hog.h` instead of leaving the main arithmetic as `float`. In `ap_fixed<W,I>`, `W` is the total width and `I` is the number of integer bits including the sign, leaving `W-I` fractional bits. In hardware this is implemented as integer arithmetic with an implied binary point, so additions are much cheaper than floating point operations. This is important for reaching $\Pi=1$ in the accumulator loops at a 5 ns clock. We chose each width to be large enough for the range it needs to store, with some headroom to avoid overflow.

Type	What it holds	Peak value it must fit	Integer range available
PIX_TYPE<48,24>	gamma-corrected pixel $x^2\sqrt{x}$	$255^{2.5} \approx 1.04 \times 10^6$	$\pm 2^{23} \approx 8.4 \times 10^6$
VAR_TYPE<64,32>	pixel sum, block and cell energies	$\approx 1.06 \times 10^9$	$\pm 2^{31} \approx 2.1 \times 10^9$
ACC_TYPE<96,64>	sum of squared pixel values	$\approx 1.10 \times 10^{15}$	$\pm 2^{63} \approx 9.2 \times 10^{18}$
VAR_MEAN_TYPE<72,40>	variance before the square root	$\leq M^2/4 \approx 2.7 \times 10^{11}$	$\pm 2^{39} \approx 5.5 \times 10^{11}$
HIST_T<32,16>	per-cell histogram bin	bounded by cell magnitudes	$\pm 2^{15}$
BIN_T = ap_uint<3>	orientation bin index	7	0 to 7

(b) The integer widths were chosen from the largest values each type can reach. A pixel is at most 255, so the gamma-corrected value $x^2\sqrt{x}$ peaks at $255^{2.5}$, just under 2^{20} ; PIX_TYPE therefore has 24 integer bits. Summing 1,024 of these values adds about ten bits, so VAR_TYPE uses 32 integer bits. The sum of squared values can reach roughly 2^{50} , which is why ACC_TYPE uses 64 integer bits. Variance is bounded by $M^2/4$ for values in $[0, M]$, putting it just under 2^{38} , so 40 integer bits are enough for VAR_MEAN_TYPE. Finally, BIN_T only needs three unsigned bits because there are exactly eight orientation bins. The expression $(k + 4) \& 7$ also relies on that three-bit wraparound.

(c) The fractional bits control precision instead of range. PIX_TYPE has 24 fractional bits, and the matching `fixed_sqrt<48,24>` implementation in `math_helper.cpp` produces one result bit per iteration at that precision. The accumulator types use 32 fractional bits so repeated additions keep enough precision. PIX_TYPE was also widened from <40,24> during development after the narrower version proved insufficient, so the final widths were based on both range calculations and testing.

(d) These type choices also matter for the later optimisations. Since the accumulators use fixed point and stay within their ranges, splitting an accumulation across multiple lanes does not change the result when the partial sums are combined. We also saw the same C-Sim correctness signature, with a maximum absolute difference of 0.00000310, across every restructure from v5 to v10. This gave us confidence that the chosen widths were not causing overflow or noticeable loss of precision.

Baseline Design Constraints

(a) The first working kernel was actually slower than the ARM version, giving a $0.79\times$ kernel speedup and $0.78\times$ end-to-end. At this stage it still used floating point and pipelining was disabled on the main loops because the pipelined versions did not fit on the device. The original implementation used `double` arithmetic and `std::pow(x, 2.5)` for gamma correction, which produced FF 206%, LUT 127% and BRAM 176%. A generic double-precision `pow` alone synthesised to roughly 78 DSPs and 11,000 FF per instance. Changing to float and rewriting the expression as `x * x * sqrt(x)` reduced the usage significantly, but LUT utilisation was still 110%. Most of the remaining LUT cost came from the histogram update `hist[bin] += weight`. Because `bin` is only known at runtime, a pipelined version needs extra hazard-handling logic when multiple iterations are in flight, which cost roughly 68,000 LUTs, or 58% of the device budget. The CORDIC used for gradient angles added another 17,700 LUTs. Disabling pipelining removed the overlap that required this forwarding logic and allowed the design to fit at LUT 31%, FF 10% and BRAM 13%, although it then ran mostly as a sequential state machine.

Design	SW (ARM)	HW kernel	Speedup	Transfer share
Baseline, un-pipelined float	420.92 μ s	533.14 μ s	0.79 $\times$	1.1%
Final, fixed-point dataflow	420.07 μ s	3.10 μ s	135.50 $\times$	4.3%

(b) Transfer overhead was only 1.1% of the baseline end-to-end time, so the main problem was computation on the FPGA rather than data movement. Batching alone would not have fixed this version. The baseline also reported an RMSE and maximum difference of 0.0, but this result needs some context: the host compared the kernel against a software model using the same arithmetic, and the test used one synthetic image rather than the CIFAR-10 dataset used later. The $0.79\times$ result should therefore be treated as a single-image baseline, not as a directly comparable dataset benchmark.

(c) We also tried keeping floating point and improving throughput within that design. The accumulator recurrences were split into eight interleaved lanes, and the dynamic histogram update was replaced with predicated updates to fixed addresses. This reached an initiation interval of one on the target loops and reduced the synthesis estimate from 190 to 43 μs while still passing C-Sim. However, it had an unresolved -0.53 ns estimated timing slack against the 5 ns target. We did not build this version to an xclbin or run it on the board, and instead moved on to the fixed-point approach.

(d) The version that eventually reached $6.16\times$ was developed from the original $0.79\times$ baseline, but replaced the main sources of cost instead of trying to keep the floating-point design. The accumulators were converted to fixed point, allowing an addition to complete within one clock cycle and giving $\text{II}=1$ without needing the eight interleaved lanes. Gradient orientation was also changed from a CORDIC arctangent to comparisons against thresholds based on $\tan(22.5^\circ)$ and $\tan(67.5^\circ)$. Since HoG only needs the orientation bin rather than the exact angle, this removed the CORDIC completely. We kept the predicated histogram updates from the earlier attempt because fixed write addresses avoid the large hazard network caused by `hist[bin] += weight`.

```

Using generated synthetic image
Opening device 0
-----
Feature length      : 288
RMSE (HW vs SW)    : 0.0000000000000000
Max abs diff       : 0.0000000000000000
-----
Averaged over 100 runs (plus 1 warm-up)
-----
Software (ARM)      :      420.92 us
-----
HW write (H2D)      :       2.79 us
HW kernel only      :      533.14 us
HW read (D2H)       :       3.41 us
HW end-to-end       :      539.34 us
-----
Speedup (kernel only) :      0.79 x
Speedup (end-to-end) :      0.78 x
Transfer overhead    :      1.1 % of end-to-end
-----
PASS: HW output matches SW reference!

```

Figure 1: Board output for the un-pipelined floating point baseline, one synthetic image per launch, giving $0.79\times$ with transfer overhead at only 1.1%.

(e) After these changes, including fixed-point arithmetic, predicated histogram updates, threshold-based orientation binning and fewer full-image passes, the kernel dropped from 533 μs per image to 68 μs per image. This gave a $6.16\times$ kernel speedup and became the starting point for the later optimisation work.

Methodology Behind the Optimisations

Once the design was pipelined and fit on the device, we used the synthesis and board results to decide what to optimise next. The same process was repeated for each later version.

(a) From $6.16\times$ onward, we followed the same basic process for each optimisation. We first measured the current design and identified the largest cost, then made one change aimed at that result. The new version was

checked with C-Sim, C-Synth, Co-Sim and, when appropriate, on the board before we moved on. The next optimisation was then chosen from the updated measurements. The table below summarises the main steps from $6.16\times$ to $135\times$.

Step	What the report showed	Action taken	Loop stage
Batched launch	Board $68.21\mu\text{s}/\text{image}$, synthesis $20.4\mu\text{s}$, a $48\mu\text{s}$ gap outside the kernel	One launch for all 10,000 images, 16-wide burst copies matched to AXI width	Check system overhead first
Block norm	1,491 cycles, largest block in the kernel	Precompute per-cell energies once instead of per block	Optimise the largest stage
Sqrt attempt	LUT 51% to 92% after inlining	Reverted same day, replaced with pipelined float sqrt	Check the resource cost
Normalise	Largest remaining stage at 1,132 cycles	Widened to two pixels per cycle, lane-split accumulators	Move to the next bottleneck
Dataflow	Six stage costs individually known	Restructured into dataflow region, interval set by slowest stage	Use the measured stage costs
Gradients	Pacing stage at 1,033 against 621 next	Widened to two pixels per cycle, six-bank partition	Optimise the pacing stage
Build check	Build marker did not match intended version	Rebuilt and re-verified before trusting the report	Confirm the correct build

(b) The first major improvement came from comparing the board timing with the synthesis estimate. The kernel took $68.21\mu\text{s}$ per image on the board, while the synthesis report estimated only $20.4\mu\text{s}$, or 4,083 cycles at 200 MHz, for the compute pipeline. The roughly $48\mu\text{s}$ difference pointed to overhead outside the kernel, with per-invocation launch cost being the main suspect. We therefore changed the host to process all 10,000 images in one launch instead of launching the kernel once per image. The first batched version was correct, but synthesis showed that its copy loops still moved only one element per cycle even though the AXI bus had been widened to 512 bits. We then unrolled the copy loops 16-wide to match the bus. After this change, the board time dropped to $20.81\mu\text{s}$ per image.

(c) After batching, the synthesis report showed that block normalisation and pixel normalisation were the two largest remaining sections at 1,491 and 1,132 cycles. We therefore worked on those stages next. One attempt to speed up block normalisation used three parallel square roots, but synthesis pushed LUT usage to 92%. We reverted that version before building it for the board and used a cheaper pipelined square-root approach instead.

(d) The move to dataflow produced one of the largest jumps, from roughly $33\times$ to $81\times$. By this point we already had cycle counts for each of the six stages, so we restructured them into a dataflow region where different images could occupy different stages at the same time. The throughput of this structure is limited by the slowest stage instead of the sum of all stage latencies. We relied heavily on Co-Sim for this change because C-Sim does not model the stages running concurrently, so some dataflow problems would not appear there. Random stall injection in Co-Sim was used before accepting the design.

(e) The last optimisation was easier to choose because the dataflow report showed the bottleneck directly. The gradient stage was taking 1,033 cycles, compared with 621 cycles for normalisation, so we widened the gradient calculation to two pixels per cycle. Table 1 shows the final stage costs. Normalisation now takes 621 cycles and gradients take 583, so the two slowest stages are reasonably close and there is no obvious next bottleneck. The table also shows that cycle count and area cost are different issues. `compute_cells` takes only 301 cycles but uses 32% of the device LUTs because it contains four parallel histogram units. This limits how much more parallelism can be added without creating an area problem.

Dataflow stage	Cycles	LUT	DSP	Role
normalize_image	621	12%	3%	sets the interval
compute_gradients	583	6%	4%	widened last
block_normalize	388	11%	10%	optimised earlier
compute_cells	301	32%	0%	low cycle count, highest LUT cost
load_image_stage	64	~0%	0%	burst read
store_feat_stage	18	~0%	0%	burst write

Table 1: Per-stage cost of the final dataflow design, taken from the C-Synth report. The stage interval of the region is set by the slowest stage rather than the sum, so the design processes one image every 622 cycles.

(f) There are two timing figures to distinguish. One image takes 2,128 cycles, or about 10.6 μ s, to travel through the complete pipeline from the first burst read to the final feature write. The 3.10 μ s per image reported elsewhere is the steady-state interval once several images are overlapping in the dataflow pipeline. This interval determines the total time for the 10,000-image batch. Dataflow improves throughput by overlapping images, but it does not reduce the full latency of a single image. Since our host submits the full dataset in one launch, the steady-state throughput is the more relevant figure. An application that processed one image at a time would instead care more about the 10.6 μ s latency.

(g) We stopped at 135 $\times$ because the remaining optimisations were starting to cost too much area. Table 2 shows how the resource usage changed across versions. The early improvements, such as batching and the block-normalisation rework, mostly changed how the existing hardware was used and added relatively little area. Later changes increased parallelism and therefore used more LUTs and other resources. The device has a hard limit at 100% utilisation, but place and route can become difficult well before that point. We had already seen this risk in the reverted square-root version, which reached 92% LUT. AMD’s methodology guidance recommends keeping enough spare fabric for routing, with roughly 75% often used as a practical upper range. The final design uses 66% LUT. Widening normalisation again was estimated to require about 12,000 more LUTs, which would move the design much closer to the point where routing becomes the main risk.

Version	Cycles/image	Speedup	LUT	DSP	FF	BRAM
v5 batched interface	4,175	20.16 $\times$	51%	4%	8%	17%
v6 3-wide sqrt, reverted	3,063	not deployed	92%	13%	20%	17%
v7 per-cell energies, float sqrt	3,058	—	60%	13%	14%	17%
v8 2-wide normalisation	2,547	—	65%	15%	15%	14%
v9 six-stage dataflow	1,034	81.47 $\times$	64%	15%	17%	40%
v10 2-wide gradients	622	135.50 $\times$	66%	17%	19%	40%

Table 2: Cost of each optimisation in fabric. Cycles per image are the C-Synth figures, which become the dataflow region interval from v9 onward. Speedups are board-measured where the version was deployed. v7 and v8 were verified through Co-Sim and folded into the v9 build rather than deployed separately.

Removing the Accumulator Dependency

(a) The image normalisation stage calculates a running sum and sum of squares over all 1,024 pixels to obtain the image mean and standard deviation. In the original loop, `sum += v[i]` creates a dependency because each iteration reads the accumulator value written by the previous iteration. With fixed-point arithmetic, one addition still completes within a clock cycle, so the loop can already achieve II=1 when processing one pixel per cycle.

(b) To process two pixels per cycle, the loop uses two accumulator lanes instead of one. One lane handles even-indexed pixels and the other handles odd-indexed pixels, as shown in Figure 2. Each lane still has its own

dependency chain, but the two chains are independent and can be updated in parallel.

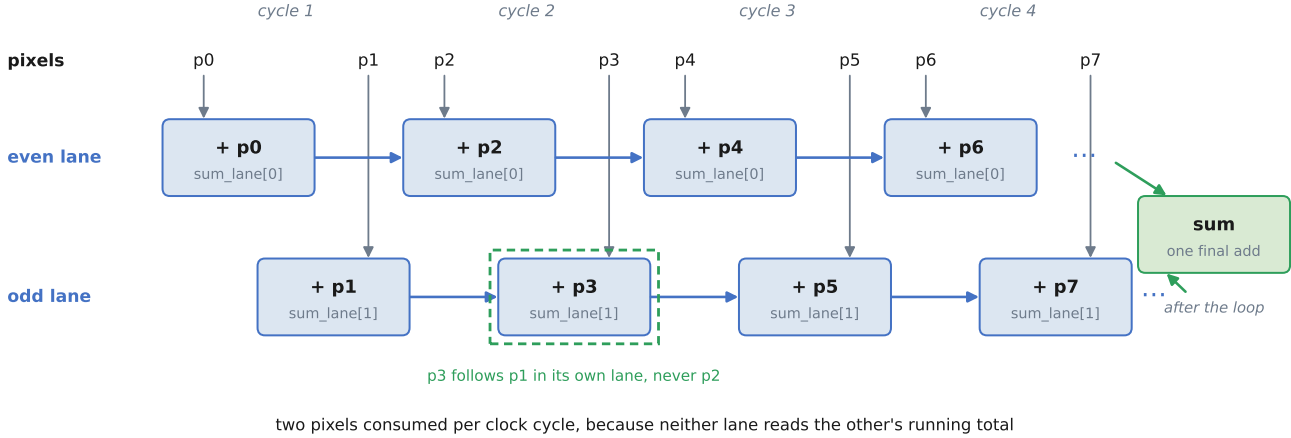


Figure 2: The accumulation split across two lanes. Each lane is still a serial chain, but the chains are independent, so one even-indexed and one odd-indexed pixel are consumed in the same clock cycle. The two running totals are combined once, after the loop.

(c) For example, pixel 3 depends on pixel 1 through `sum_lane[1]`, while pixel 2 uses `sum_lane[0]`. Since the lanes do not access each other's state inside the loop, the two additions can happen in the same cycle. After the loop, the two partial sums are added together once. Because the fixed-point values stay within range, regrouping the additions does not change the result. We used the same lane-splitting idea for the per-cell energy accumulation in block normalisation and in the two-pixel-wide gradient stage.

Correctness Methodology and Tolerance

(a) Each board run is checked in two ways. First, the FPGA output is compared with an internal C++ model that uses the same arithmetic as the ARM software path. Second, both the hardware and software outputs are compared with `expected_keyed.txt`, which was generated separately from a reference HoG implementation and is indexed by filename. This gives an independent reference for all 10,000 images. Figure 3 shows the full check completing with every image matched.

Comparison	RMSE	Max abs diff	Images outside tol.
SW vs independent reference	0.000011535165119	0.009568840265274	0
HW vs independent reference	0.000025413844672	0.015522837638855	2

(b) The software output has zero images outside tolerance and matches the independent reference very closely. The hardware output has two images outside the $1e-2$ tolerance, although the overall RMSE is still very small. The difference comes from orientation binning. The hardware uses fixed-point comparisons against thresholds based on $\tan(22.5^\circ)$ and $\tan(67.5^\circ)$, while the reference uses a floating-point arctangent. In the two affected images, one gradient lies close enough to a bin boundary that the hardware and reference place it in neighbouring bins. This moves one weighted vote from one histogram bin to the next, which explains why the maximum difference of 0.0155 is much larger than the overall RMSE.

(c) We used a $1e-2$ board tolerance instead of the $1e-4$ tolerance used in C-Sim because of this known bin-boundary difference. Each 288-value descriptor is L2-normalised per block, so moving one pixel's vote between adjacent bins only changes a small part of the final descriptor. The same two images exceed the tolerance in

every kernel version, which suggests this is a repeatable input-dependent boundary case rather than an unstable hardware error.

```

Times exclude file loading and one warm-up launch
HW mode      : single batched launch (10000 images)
-----
Software total      : 4200.69 ms
Software avg/image  : 420.07 us
Software throughput  : 2380.56 images/s
-----
HW total end-to-end : 32.39 ms
HW write avg/image  : 0.07 us
HW kernel avg/image : 3.10 us
HW read avg/image   : 0.07 us
HW end-to-end avg   : 3.24 us
HW throughput       : 308723.97 images/s
-----
Speedup (kernel only) : 135.50 x
Speedup (end-to-end)  : 129.69 x
Transfer overhead     : 4.3 % of end-to-end
-----
Independent reference cross-check (/home/petalinux/expected_keyed.txt)
-----
Images cross-referenced : 10000
Images missing from ref : 0
-----
SW vs reference RMSE : 0.000011535165119
SW vs reference max diff : 0.009568840265274
SW images outside tol. : 0
-----
HW vs reference RMSE : 0.000025413844672
HW vs reference max diff : 0.015522837638855
HW images outside tol. : 2
-----
FAIL: 2 image(s) do not match SW within 1e-02
xilinx-kv260-starterkit-20251:~$

```

Figure 3: Board output for the full 10,000 image run, showing both the HW-vs-SW comparison and the independent reference cross-check.

Future Improvements

(a) During a run, all 10,000 images are stored in off-chip DDR memory and are burst-read by the kernel through its AXI master port. The 40% block RAM utilisation reported for the FPGA does not mean that the dataset is stored on chip. That BRAM is used for the per-image working buffers inside the pipeline, including the image tile, gamma image, bin map, magnitudes and cell histograms. These buffers are also duplicated where ping-pong channels are needed between dataflow stages. As a result, the kernel only keeps a small number of images’ worth of data on chip at any one time.

(b) One possible improvement is to stage several images in on-chip memory before they are processed, instead of reading every image from DDR immediately before use. This could reduce the effect of DDR latency. The current design uses 117 of the 288 available 18 Kb block RAMs. Using AMD’s roughly 75% utilisation guideline as a practical limit gives a budget of 216 blocks, leaving 99 blocks, or about 1.78 Mb, available for extra staging buffers.

Block RAM budget	18 Kb blocks	Capacity
Available on the XCK26	288	5.06 Mb
Used by the current design	117 (40%)	—
Ceiling at AMD’s 75% guidance	216	—
Spare for image staging	99	1.78 Mb
Images at 1,024 bytes each, packed densely	—	about 220 images
Images if staged with the same 16-way partitioning as the current input buffer	—	about 13 images

(c) The number of images that can be staged depends heavily on how the buffer is accessed. Array partitioning creates separate memory banks for parallel access, so a partitioned buffer can use much more BRAM than its raw data size suggests. For example, `img_local` stores one 1,024-byte image but uses 16 block RAMs because it is cyclically partitioned 16 ways for the 16-wide burst copy and is also duplicated for ping-pong buffering. If images were packed densely, one 18 Kb block could hold about two images, so the partitioned form uses roughly

sixteen times more memory than the raw image data requires. A sequential staging buffer could therefore get much closer to the dense estimate in the table, while a highly parallel buffer would be closer to the lower estimate.

(d) UltraRAM is currently unused in every version of the design, so it could provide additional on-chip storage beyond the BRAM budget above. However, transfer overhead is already only 4.3% of the end-to-end runtime, which limits the maximum benefit of extra staging. Any staging design would therefore need to be tested with several buffer sizes, because a larger buffer uses more resources and adds fill latency while only targeting a relatively small part of the total runtime.

Use of AI

(a) AI tools, mainly Claude, were used during the optimisation stage of the project. The HoG algorithm and the first working HLS implementation had already been developed by the team. We mainly used AI to discuss possible optimisations, help interpret synthesis and debugging results, and generate supporting code such as the timing and correctness reporting in the host application. This supporting code was separate from the main HoG implementation being evaluated.

(b) AI was particularly useful for debugging. For example, it helped us trace a Co-Sim segmentation fault to interface `depth` pragmas that did not match the testbench buffer allocation. The Co-Sim wrapper copies the number of elements specified by `depth`, even if the kernel itself uses fewer. It also helped explain the large LUT increase in the reverted square-root attempt, where auto-pipelining the outer loop caused nested loops to be fully unrolled. In both cases, the relevant evidence was already in our reports and AI helped us interpret it.

(c) We also used AI to discuss possible optimisations after the pipelined baseline was working. The main techniques we applied were already covered in the course, including loop unrolling, pipelining and array partitioning. We also used the Chapter 8 discussion in *Parallel Programming for FPGAs* that dataflow throughput is limited by the slowest stage rather than the sum of all stage latencies. AI was useful for discussing where these techniques could be applied in our design and how to implement them in HLS.

(d) We did not assume AI suggestions were correct. Any suggested change still had to pass the same C-Sim, C-Synth, Co-Sim and board checks as the rest of the design. Some suggestions were rejected after testing. For example, the three-wide square-root version reached 92% LUT utilisation, so we reverted it. Only changes that passed the normal verification process were kept.

Summary

The final design achieves a kernel speedup of $135.50\times$ and an end-to-end speedup of $129.69\times$ over the ARM baseline, processing all 10,000 images in 32.39 ms. The first working FPGA version was slower than the ARM implementation, so most of the improvement came from a series of smaller changes guided by synthesis and board measurements. After each change, we checked correctness before moving on to the next bottleneck. Every version from the first batched kernel to the final design was tested against the independent references across the full dataset, and the correctness results stayed consistent throughout.

References

- AMD, *Kria KV260 Vision AI Starter Kit Data Sheet and User Guide*, 2024. Available: <https://www.amd.com/en/products/system-on-modules/kria/k26/kv260-vision-starter-kit.html>
- AMD, *Vivado Design Methodology Guide: Addressing Congestion*, UG949 (v2024.2), 2024. Available: <https://docs.amd.com/r/2024.2-English/ug949-vivado-design-methodology/Addressing-Congestion>

R. Kastner, J. Matai, and S. Neuendorffer, *Parallel Programming for FPGAs*, arXiv:1805.03648 [cs.AR], May 2018. Available: <https://github.com/KastnerRG/pp4fpgas>

O. Diessel, *COMP4601 Lecture Notes*, University of New South Wales, Term 26T2, 2026.